



Informed search algorithms

Lecturer: Nguyen Thanh Binh





Outline

- *Best-first search*
 - *Greedy best-first search*
 - A^* *search*
 - *Heuristics*
- 

Tree Search

- A search strategy can be determined by an order of node expansion:

```
function TREE-SEARCH(problem, fringe) returns a solution, or failure
```

```
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
```

```
  loop do
```

```
    if EMPTY?( fringe) then return failure
```

```
    node ← REMOVE-FIRST(fringe)
```

```
    if GOAL-TEST[problem] applied to STATE[node] succeeds  
      then return SOLUTION(node)
```

```
    fringe ← INSERT-ALL(EXPAND(node, problem), fringe)
```

```
function EXPAND(node, problem) returns a set of nodes
```

```
  successors ← the empty set
```

```
  for each (action, result) in SUCCESSOR-FN[problem](STATE[node]) do
```

```
    s ← a new NODE
```

```
    STATE[s] ← result
```

```
    PARENT-NODE[s] ← node
```

```
    ACTION[s] ← action
```

```
    PATH-COST[s] ← PATH-COST[node] + STEP-COST(STATE[node], action, result)
```

```
    DEPTH[s] ← DEPTH[node] + 1
```

```
    add s to successors
```

```
  return successors
```

Best-first search

- Using an evaluation function $f(n)$ for each node:
 - Estimate of “desirability”
 - Expand the most desirable unexpanded node.
- Order all nodes in fringe in decreasing order of desirability.
- Special cases: greedy best-first search, A^* search,...

Romanian traveling problem

- How to go from Arad to Bucharest efficiently?

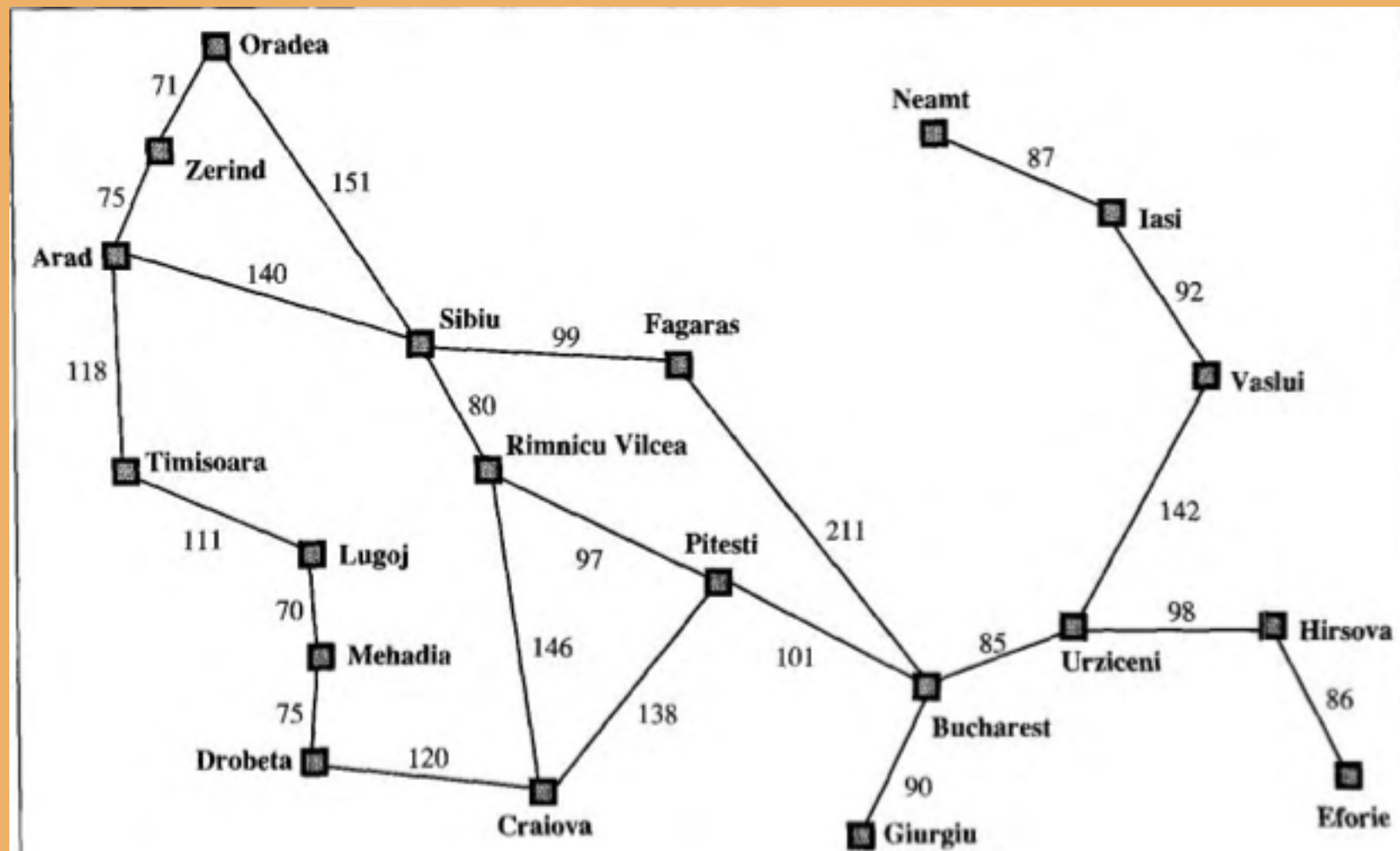


Figure 3.2 A simplified road map of part of Romania.

Romanian traveling problem

- How to go from Arad to Bucharest efficiently?

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 4.1 Values of h_{SLD} —straight-line distances to Bucharest.



Greedy best-first search

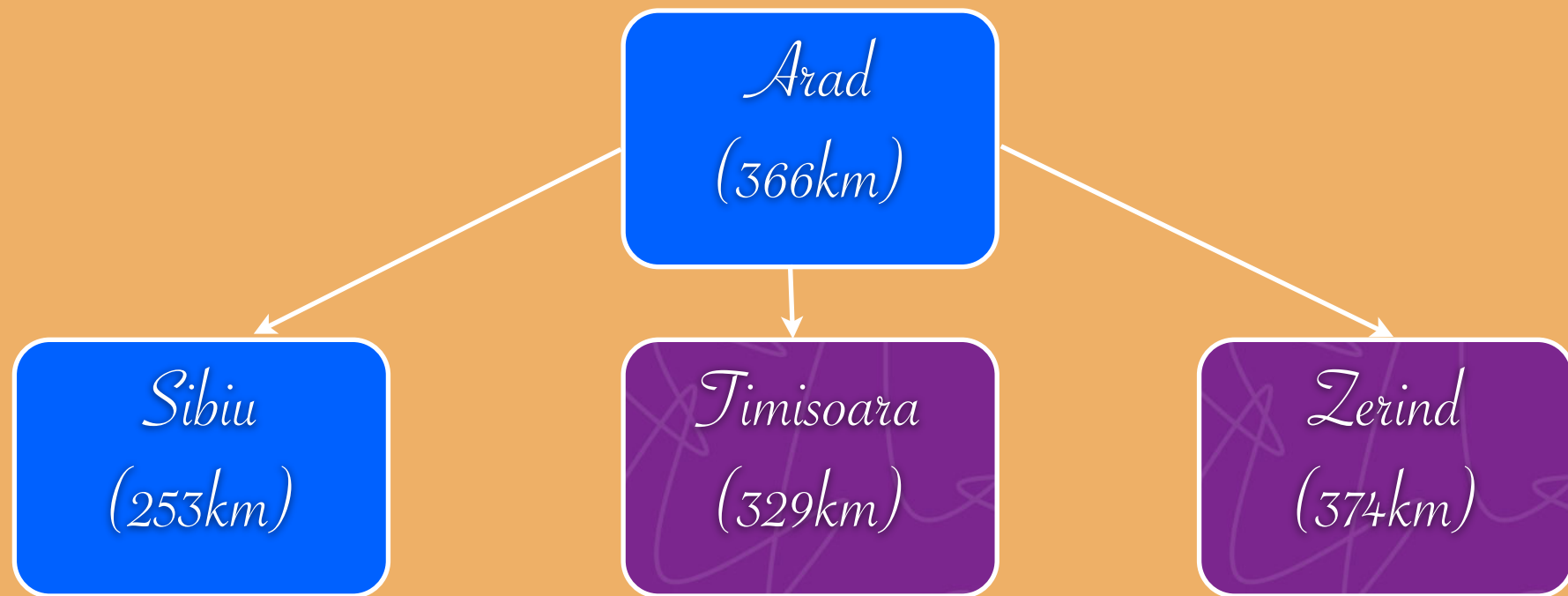
- Evaluation function: $f(n) = h(n)$ (heuristic function)
- It can estimate the cost from step n to our goal.
- For example: one can choose $h_SLD(n)$ = the straight-lien distance from the city at step n to Bucharest.
- Greedy best-first search expands the node that appears to be closest to goal.



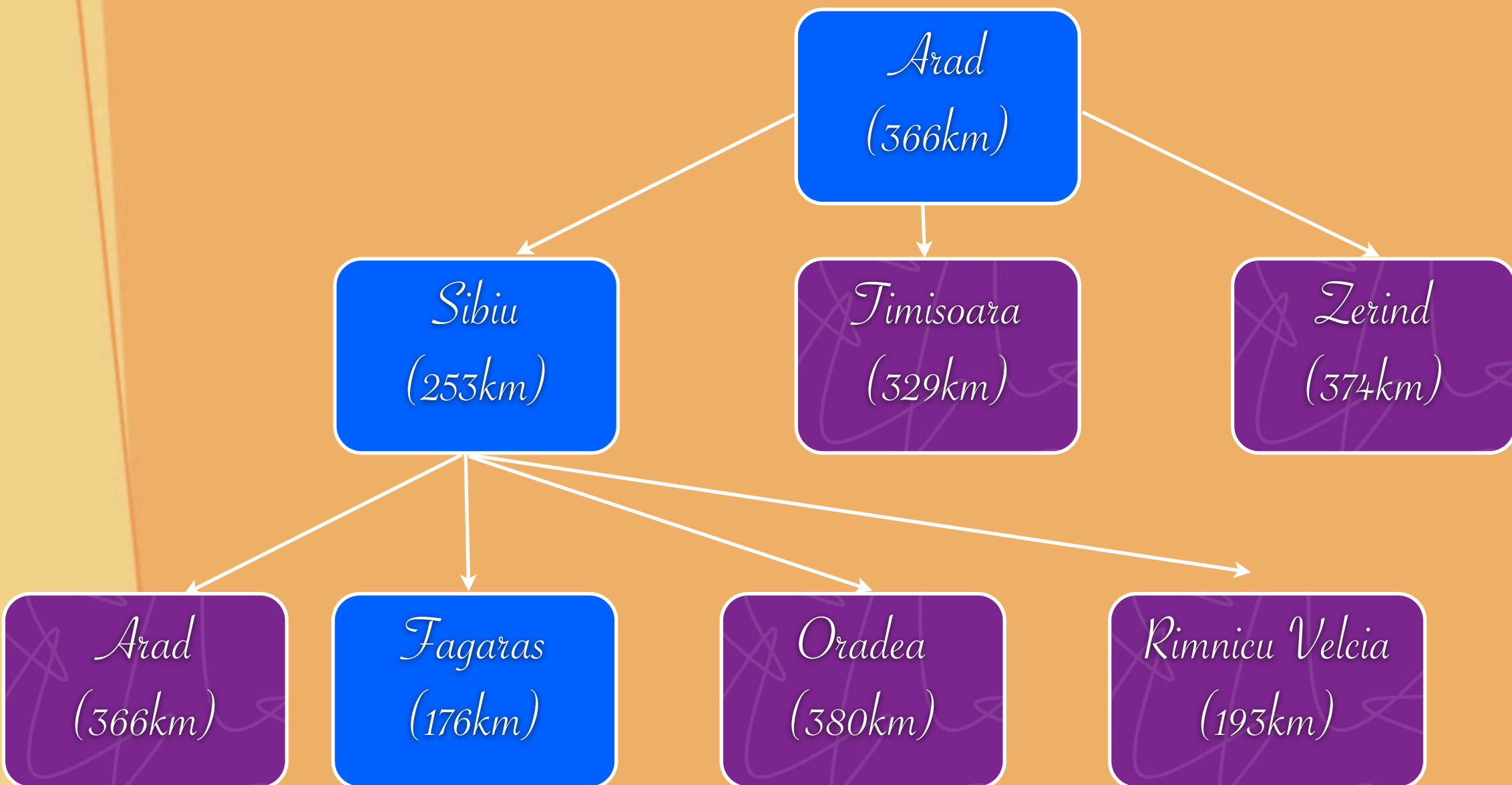
Example

Arad
(366km)

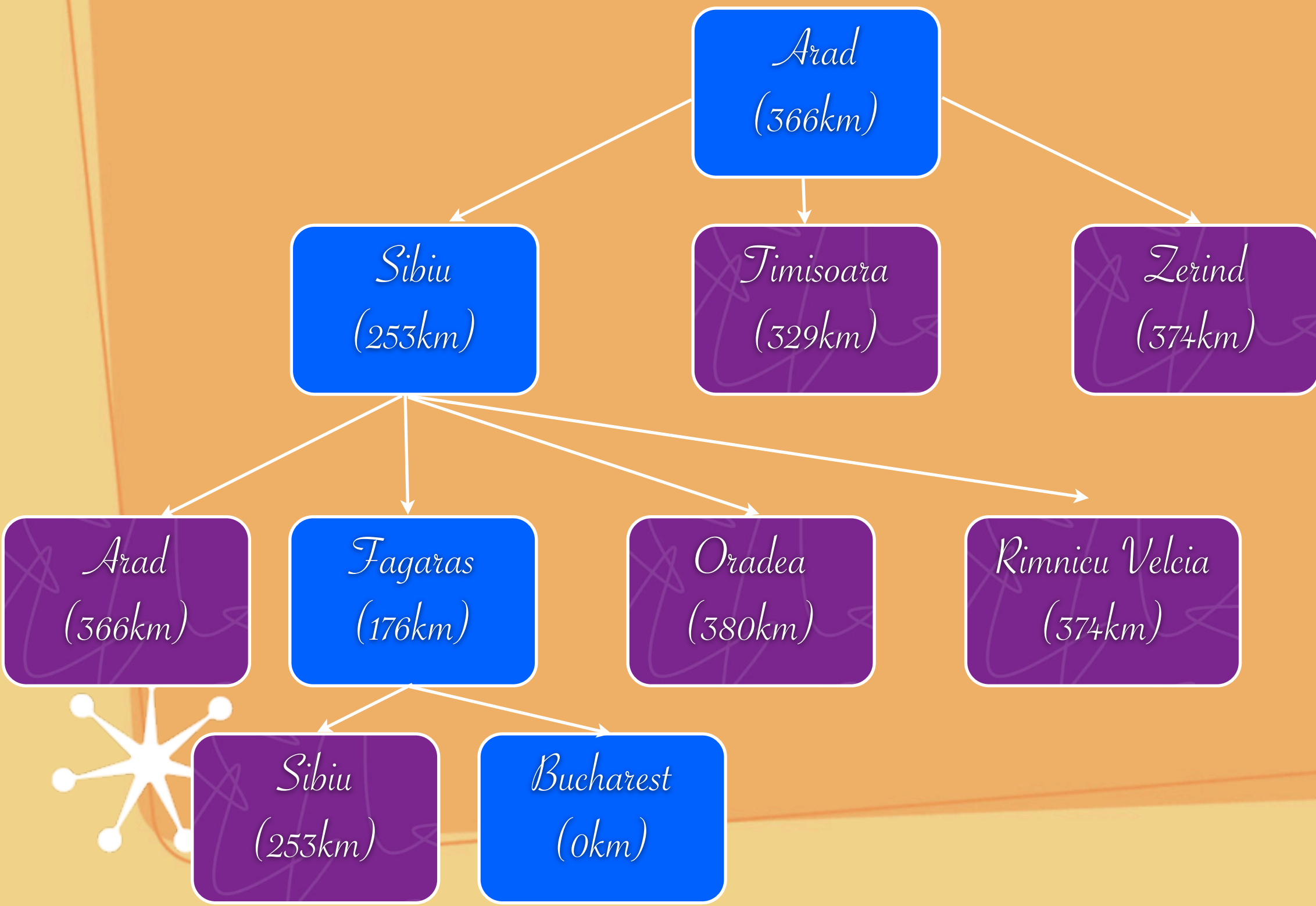
Example



Example



Example



Properties of greedy best-first search

- Complete?
- Time?
- Space?
- Optimal?

Homework 1

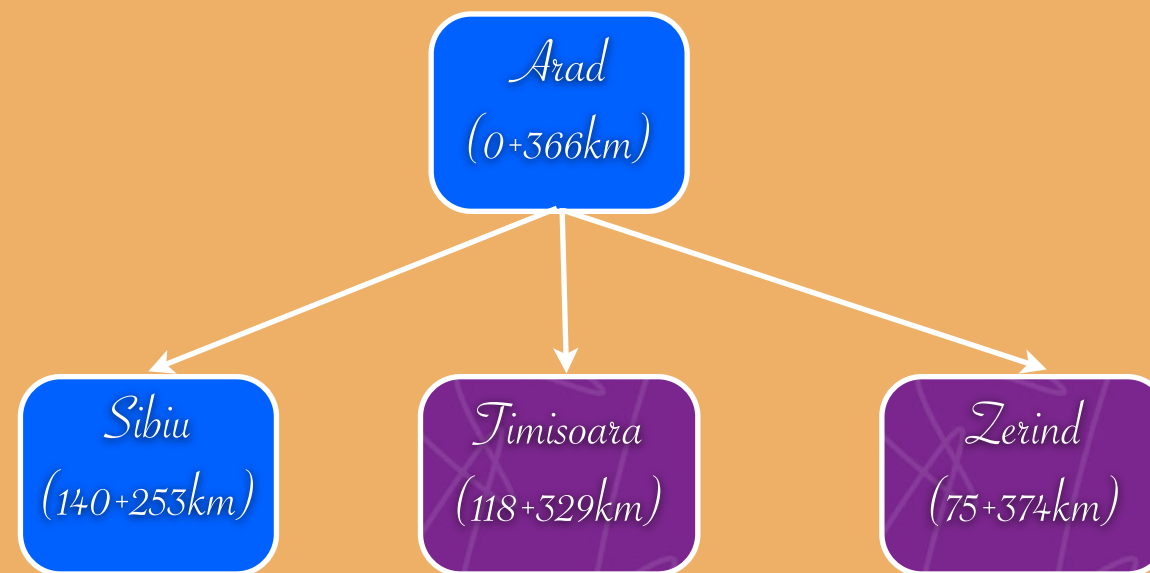
A^* Search

- Avoid expanding paths that are already expensive.
- Minimize the total estimated solution cost.
- Evaluate nodes by: $f(n) = g(n) + h(n)$
 - $g(n)$ = the path cost from the start node to node n .
 - $h(n)$ = the estimated cost of the cheapest path from node n to the goal.
 - $f(n)$ = estimated cost of the cheapest solution through n

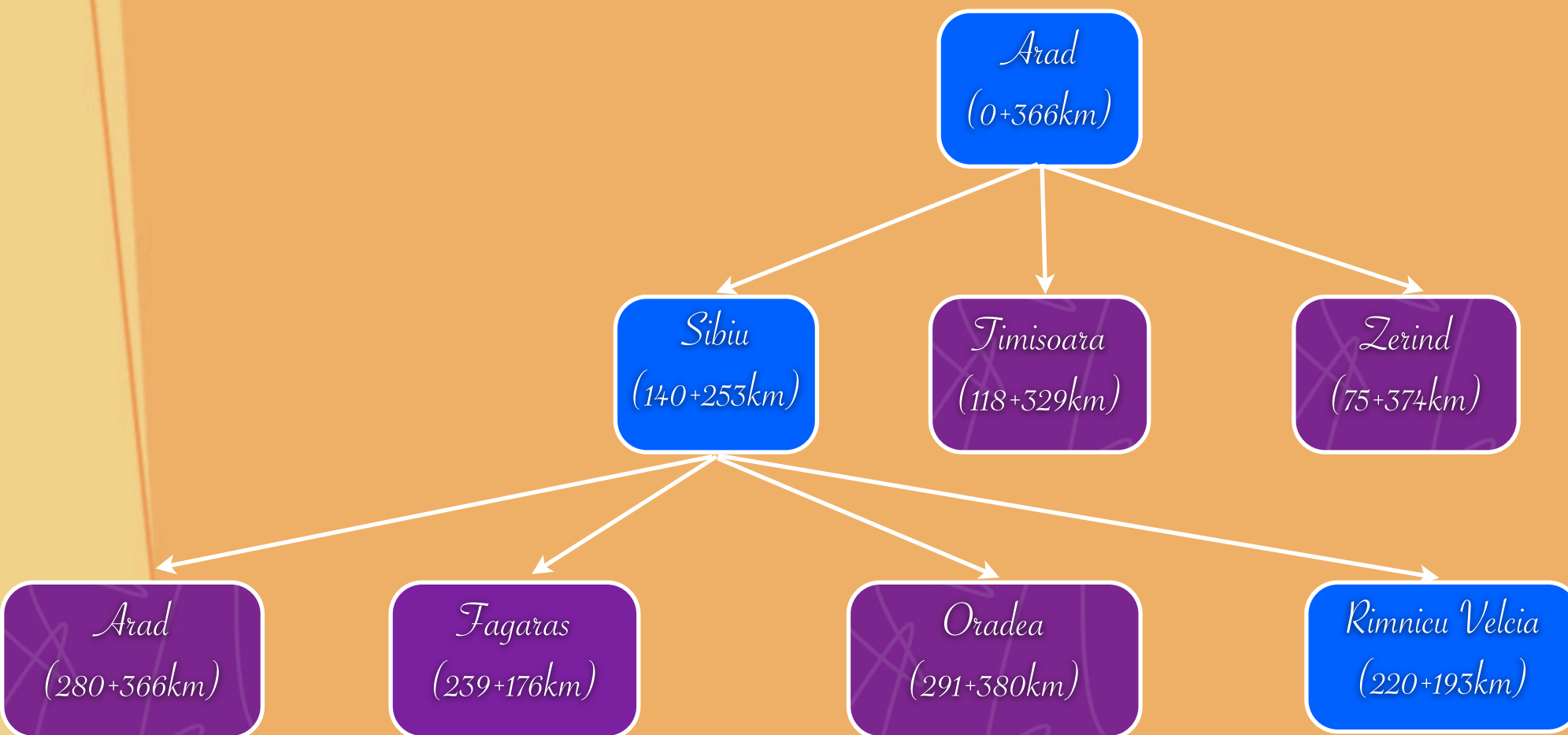
A^* search example

Arad
(0+366km)

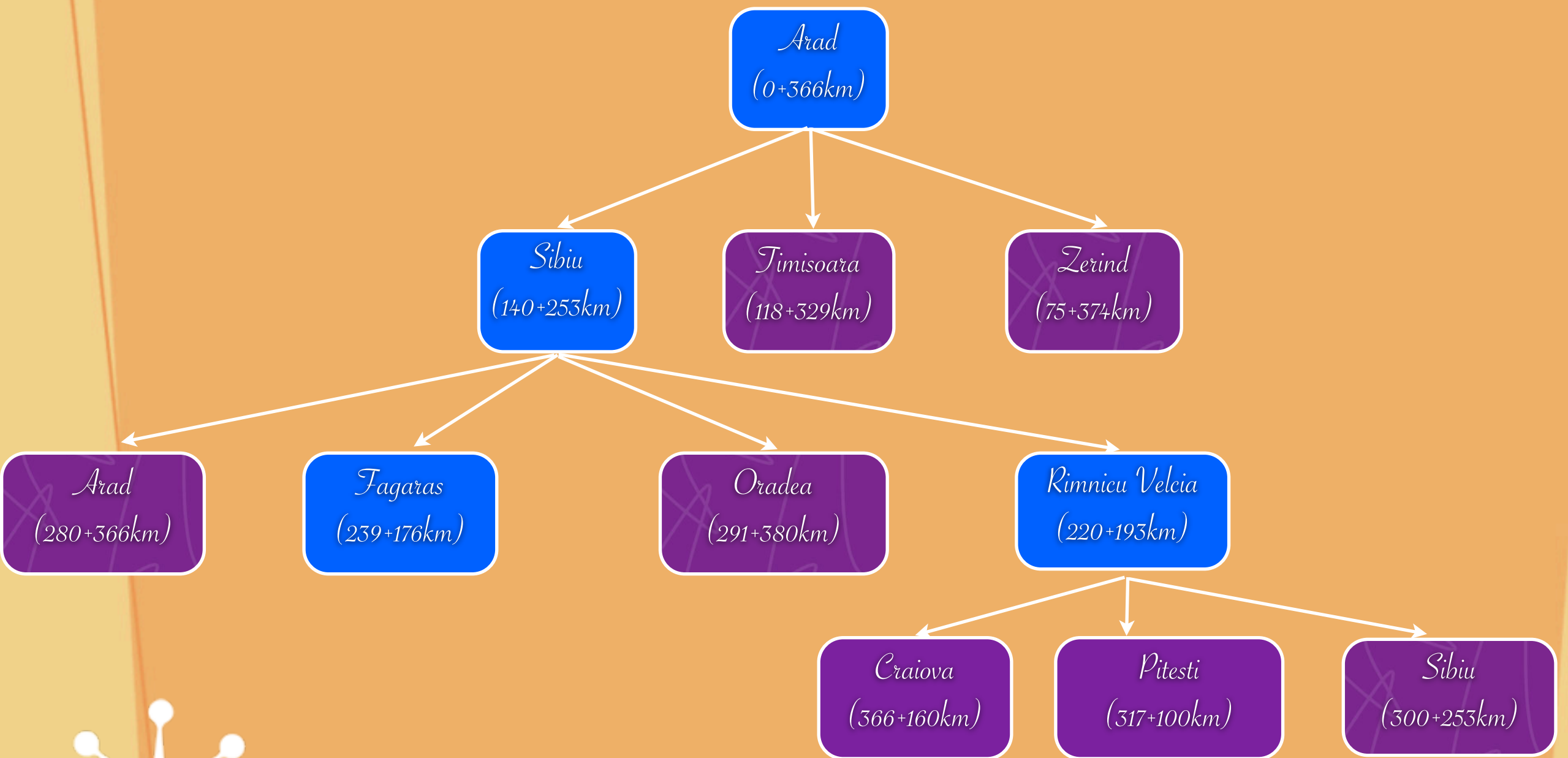
A^* search example



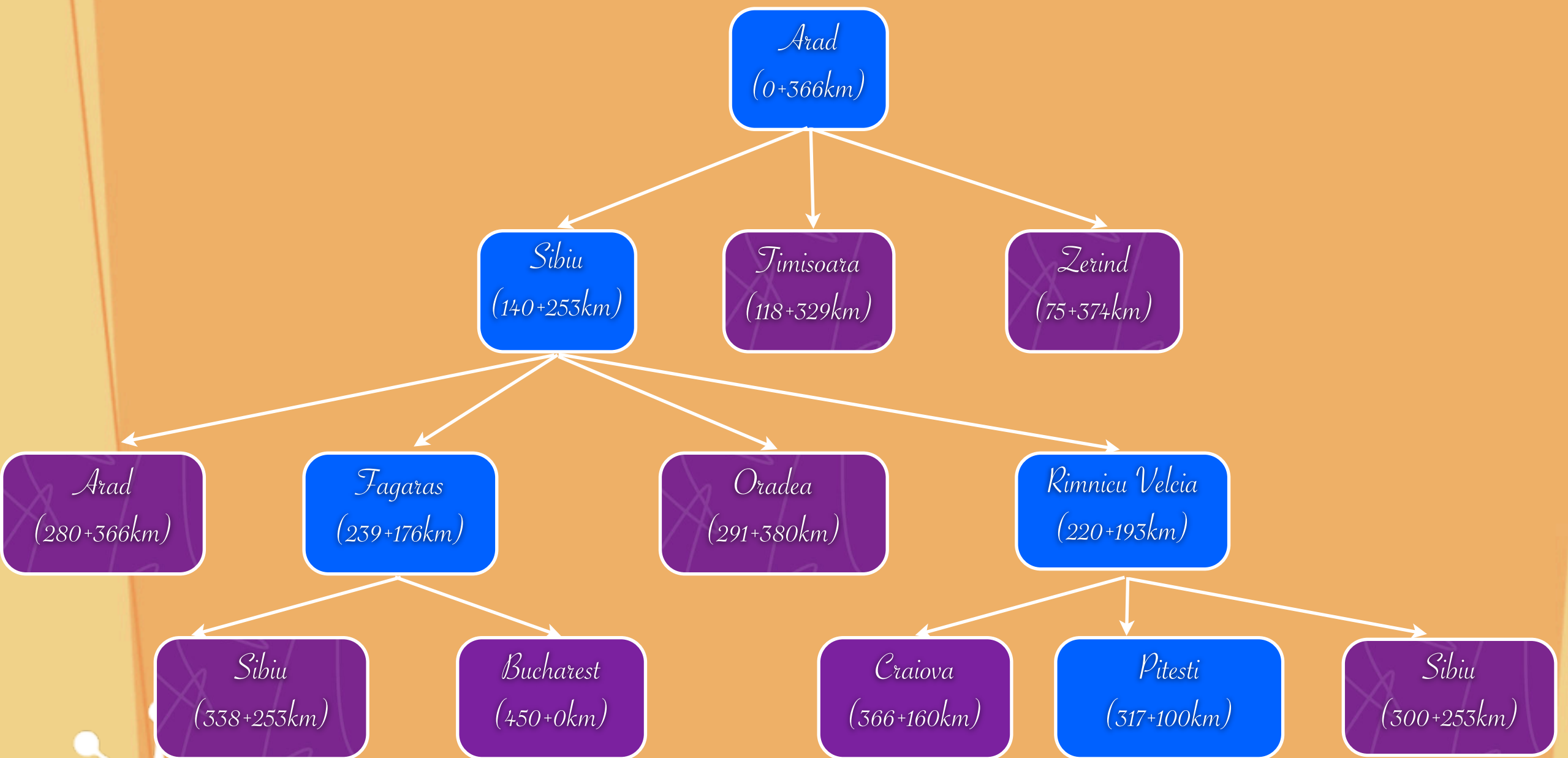
A^* search example



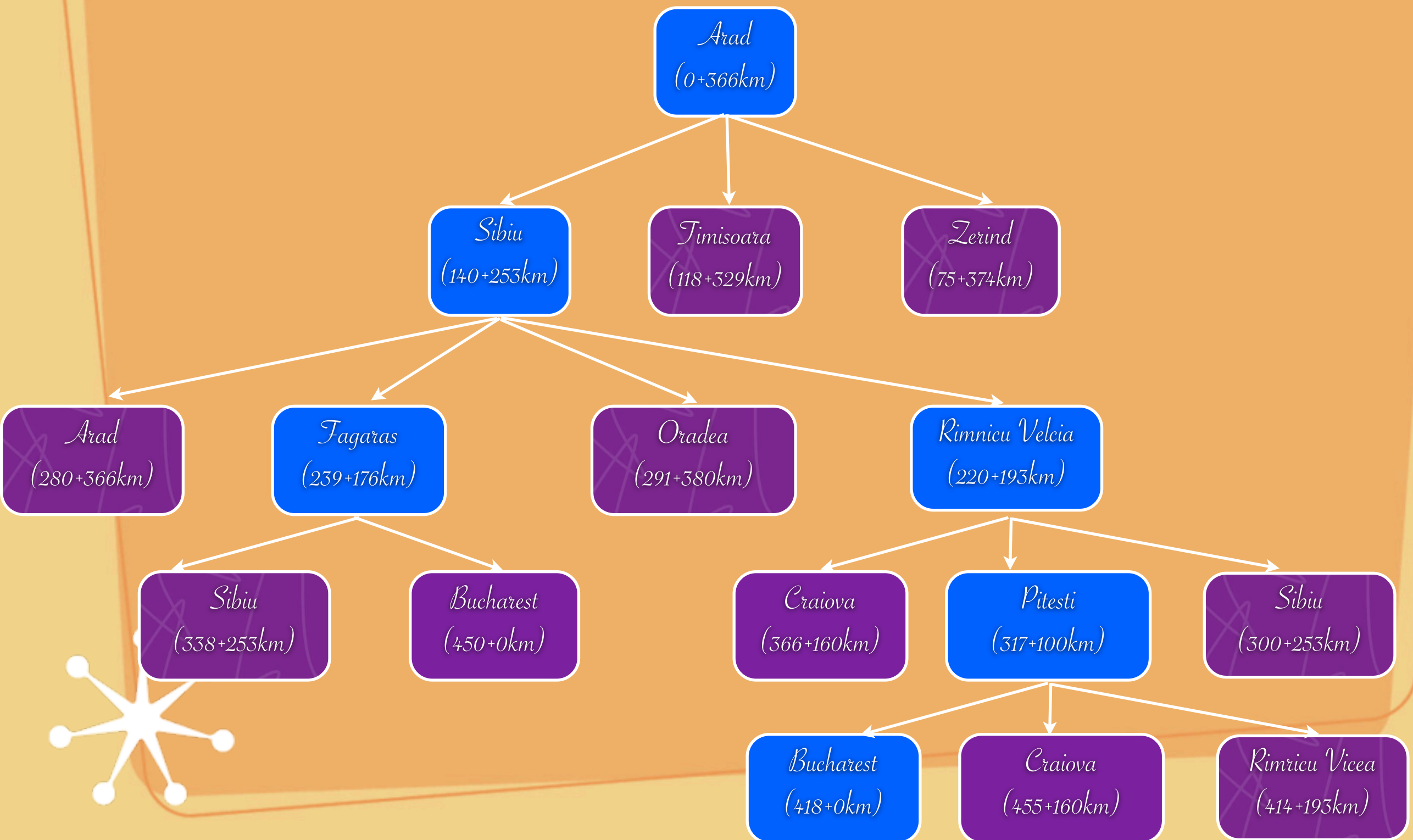
A^* search example



A^* search example



A^* search example



Admissible heuristics

- A heuristic $h(n)$ is admissible if for every node n , $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal state from n .
- An admissible heuristic never overestimates the cost to reach the goal, i.e., it is optimistic.
- Example: $h_SLD(n)$ (never overestimates the actual road distance).
- Since $g(n)$ is the exact cost to reach n , $f(n)$ never overestimates the true cost of a solution through n .

Admissible heuristics

- Theorem: if $h(n)$ is admissible, A^* using Tree-Search is optimal.
- Proof: (homework 2)
 - Suppose a suboptimal goal node G_2 appears on the fringe and let the cost of the optimal solution be C^* . Prove that $f(G_2) > C^*$.
 - Consider a fringe node n that is on an optimal solution path. Prove that $f(n) < C^*$.
 - So, G_2 will not be expanded and A^* must return an optimal solution.

Admissible heuristics

- *If we use the Graph-Search instead of Tree-Search, the proof may fail.*
- *Homework 3: Exercise 4.4 (book)*



Consistent heuristics

■ A heuristic is consistent if for every node n , every successor n' of n generated by an action “ a ”,
$$h(n) \leq c(n, a, n') + h(n')$$

■ If h is consistent, we have:

$$\begin{aligned} f(n') &= g(n') + h(n') = g(n) + c(n, a, n') + h(n') \\ &\geq g(n) + h(n) = f(n). \end{aligned}$$

■ i.e., $f(n)$ is non-decreasing along any path.





Consistent heuristics

- Theorem: If $h(n)$ is consistent, A^* using Graph-Search is optimal.
- Proof: (Homework 4).





Properties of A^* Search

- Complete?

- Time?

- Space?

- Optimal?

(Homework 5)



Admissible heuristics

- Examples: the 8-puzzle:

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State



- $h_1(n)$ = number of misplaced tiles.
- $h_2(n)$ = total Manhattan distance (i.e., the number of squares from desired location of each tile).
- $h_1(S) = ?$
- $h_2(S) = ?$

